

Basic SQL Exercise 2

2023141220023 易治行

This exercise does not provide schema-creation or data-insertion files. You may write the SQL statements directly, or create a schema and sample data first.

Part 1 University Schema Data Modifications

1. Write the following inserts, deletes or updates in SQL, using the university schema.

a. Increase the salary of each instructor in the Comp. Sci. department by 10%.

```
UPDATE instructor
SET salary = salary * 1.10
WHERE dept_name = 'Comp. Sci.';
```

b. Delete all courses that have never been offered (that is, do not occur in the section relation).

```
DELETE FROM course AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM section AS s
    WHERE s.course_id = c.course_id
);
```

c. Insert every student whose tot_cred attribute is greater than 100 as an instructor in the same department, with a salary of \$10,000.

```
INSERT INTO instructor (ID, name, dept_name, salary)
SELECT ID, name, dept_name, 10000
FROM student
WHERE tot_cred > 100;
```

Constraint note: In the supplied university schema, salary must be greater than 29000. Therefore, the requested salary of 10000 causes this statement to be rejected unless that check constraint is changed.

d. create a new course "CS-001", titled with "Weekly Seminar", with 0 credits.

```
INSERT INTO course (course_id, title, dept_name, credits)
VALUES ('CS-001', 'Weekly Seminar', 'Comp. Sci.', 0);
```

Constraint note: In the supplied university schema, credits must be greater than 0. Therefore, the requested value 0 causes this statement to be rejected unless that check constraint is changed.

e. Create a section of this course in Fall 2009, with sec_id of 1.

```
INSERT INTO section (course_id, sec_id, semester, year)
VALUES ('CS-001', '1', 'Fall', 2009);
```

f. Enroll every student in the Comp. Sci. department in the above section.

```
INSERT INTO takes (ID, course_id, sec_id, semester, year, grade)
SELECT ID, 'CS-001', '1', 'Fall', 2009, NULL
FROM student
WHERE dept_name = 'Comp. Sci.';
```

g. Delete enrollments in the above section where the student's name is Chavez.

```
DELETE FROM takes AS t
WHERE t.course_id = 'CS-001'
  AND t.sec_id = '1'
  AND t.semester = 'Fall'
  AND t.year = 2009
  AND t.ID IN (
    SELECT s.ID
    FROM student AS s
    WHERE s.name = 'Chavez'
  );
```

h. Delete the course CS-101. What will happen if you run this delete statement without first deleting offerings (sections) of this course.

```
DELETE FROM course
WHERE course_id = 'CS-101';
```

Result: section.course_id uses ON DELETE CASCADE, so course offerings and their dependent teaches and takes rows would be removed automatically. However, CS-101 is also referenced as prereq.prereq_id, whose foreign key has no ON DELETE action. In the supplied data, the statement is therefore rejected until those prerequisite references are deleted or updated. Without cascading section deletion, existing sections would also block the delete.

i. Delete all takes tuples corresponding to any section of any course with the word "database" as a part of the title; ignore case when matching the word with the title.

```
DELETE FROM takes AS t
WHERE EXISTS (
  SELECT 1
  FROM course AS c
  WHERE c.course_id = t.course_id
    AND LOWER(c.title) LIKE '%database%'
);
```

Part 2 Employee Database Queries

2. Consider the following employee database

```
employee(employee_name, street, city)
works(employee_name, company_name, salary)
company(company_name, city)
manages(employee_name, manager_name)
```

Give an expression in SQL for each of the following queries.

a. Find the names and cities of residence of all employees who work for "First Bank Corporation".

```
SELECT e.employee_name, e.city
FROM employee AS e
JOIN works AS w
  ON w.employee_name = e.employee_name
WHERE w.company_name = 'First Bank Corporation';
```

b. Find the names, street addresses, and cities of residence of all employee who works for "First Bank Corporation" and earn more than \$10,000.

```
SELECT e.employee_name, e.street, e.city
FROM employee AS e
JOIN works AS w
  ON w.employee_name = e.employee_name
WHERE w.company_name = 'First Bank Corporation'
  AND w.salary > 10000;
```

c. Find all employees in the database who do not work for "First Bank Corporation".

```
SELECT e.employee_name
FROM employee AS e
WHERE NOT EXISTS (
  SELECT 1
  FROM works AS w
  WHERE w.employee_name = e.employee_name
    AND w.company_name = 'First Bank Corporation'
);
```

d. Find all employees in the database who earn more than each employee of "Small Bank Corporation".

```
SELECT DISTINCT w.employee_name
FROM works AS w
WHERE w.salary > ALL (
  SELECT sb.salary
  FROM works AS sb
  WHERE sb.company_name = 'Small Bank Corporation'
);
```

e. Assume that the companies may be located in several cities. Find all companies located in every city in which "Small Bank Corporation" is located.

```
SELECT DISTINCT c.company_name
FROM company AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM company AS sb
    WHERE sb.company_name = 'Small Bank Corporation'
    AND NOT EXISTS (
        SELECT 1
        FROM company AS c2
        WHERE c2.company_name = c.company_name
        AND c2.city = sb.city
    )
);
```

f. Find the company that has the most employees.

```
SELECT w.company_name
FROM works AS w
GROUP BY w.company_name
HAVING COUNT(DISTINCT w.employee_name) >= ALL (
    SELECT COUNT(DISTINCT w2.employee_name)
    FROM works AS w2
    GROUP BY w2.company_name
);
```

g. Find those companies whose employees earn a higher salary, on average, than the average salary at "First Bank Corporation".

```
SELECT w.company_name
FROM works AS w
GROUP BY w.company_name
HAVING AVG(w.salary) > (
    SELECT AVG(f.salary)
    FROM works AS f
    WHERE f.company_name = 'First Bank Corporation'
);
```